Flood Mapping

Core technical design of `solution.ipynb`

Data and splitting

- The notebook loads 100 Sen1Floods11 hand-labeled patches: VV in dB from band 0, labels 1 = water, 0 = land, -1 = nodata. VH is not used.
- It downloads the first 100 patches listed in the official train CSV from the public GCS bucket, or reuses a local or Drive copy at `DATA_DIR`.
- File names are sorted, and a seeded random split at patch level gives 60 train, 20 val and 20 test.
- Validation asserts that labels are in {-1, 0, 1}, that image and label shapes match, that the VV median is negative (so the data is dB, not linear), and that every patch is more than 50% valid.

Classical filters (Median, Lee, Frost)

- Each filter runs on linear intensity, because speckle is multiplicative in that domain, and the result is converted back to dB. Nodata pixels are filled with the image median for filtering and then masked again.
- The window is 5×5 for all three.
- **Lee:** weight `w = clip(1 − cu²/ci², 0, 1)` with `cu² = 1/4.4` and `ci²` the local squared coefficient of variation. Output is `mean + w·(x − mean)`.
- **Frost:** distance-weighted average with weights `exp(−2·ci²·d)` over the window.
- **Median:** a plain median filter.

Learned denoiser

- It reuses the reference project's `ConvAE`, a 6-layer stride-1 conv autoencoder with 32 embedding channels, plus its [-30, 0] dB normalisation to [0, 1], Adam at 1e-3, and seed 0.
- A plain reconstruction autoencoder would learn the identity and keep the speckle. I therefore trained it Noise2Void-style: 2% of pixels in each 64×64 crop are replaced by a random neighbour within ±2 px, never the pixel itself, and the MSE is computed only at those pixels.
- Training uses 3,000 crops from train-split images only, for 20 epochs. A validation Noise2Void loss on val crops is printed during training. No clean images or labels are used.
- At inference the network runs unmasked on the full image.

Segmentation

- Water is `dB < threshold` on the raw or denoised image, evaluated only where the label is not -1 and the image is finite.
- Two thresholds are reported. One is the fixed -17 dB from the reference project. The other is tuned per method on the 20 val patches by maximising pooled F1 over a grid of -26 to -8 dB in 0.5 dB steps. Both are scored on the test patches.
- IoU and F1 come from pooled TP, FP and FN counts.

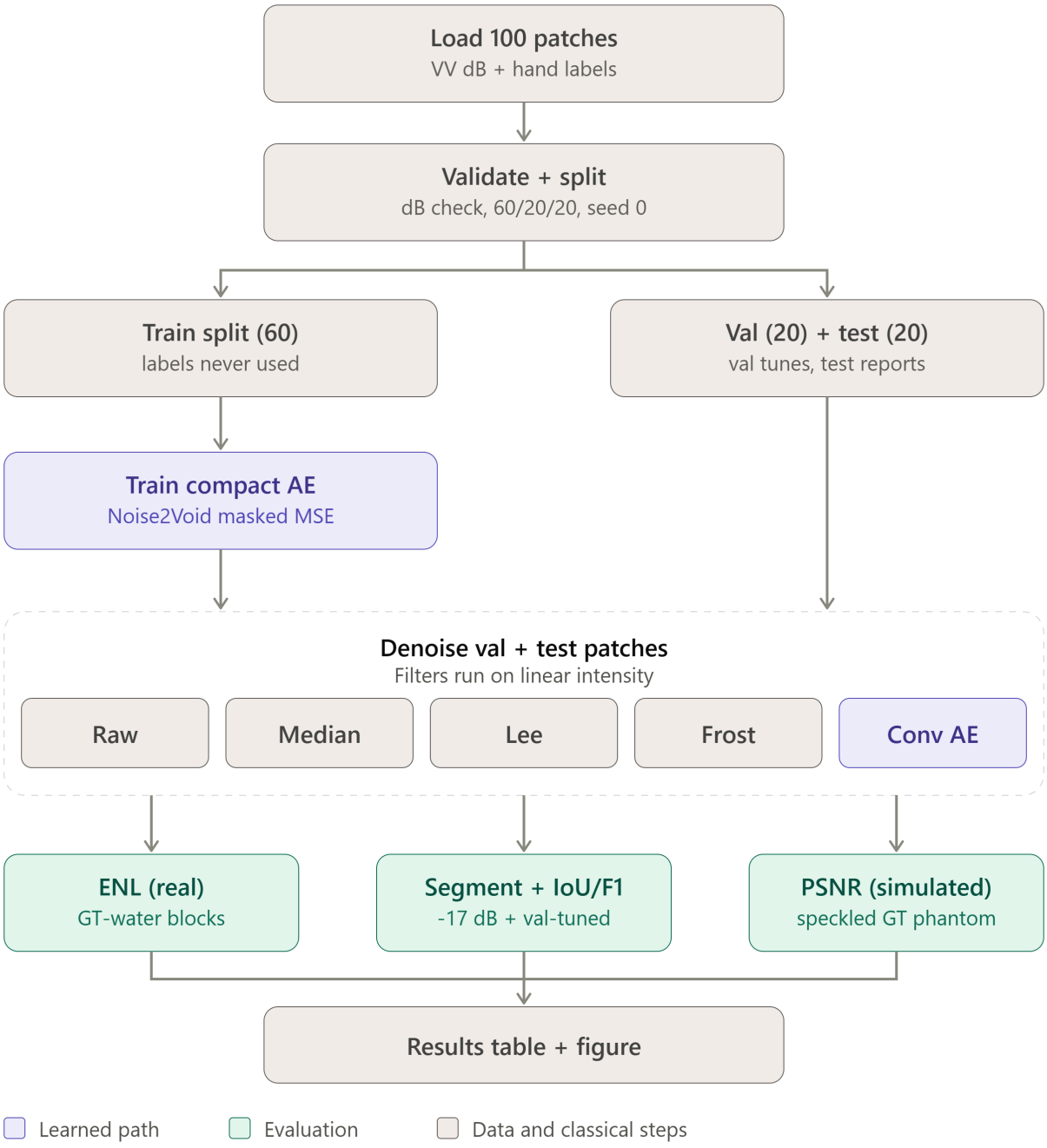
Metrics

- **ENL (real data):** the median of `mean²/var` of linear intensity over 16×16 blocks that lie entirely inside ground-truth water, on val and test patches. Higher means smoother, so it is read together with IoU and F1.
- **PSNR (simulated):** real SAR has no clean reference. For each test patch I build a two-class phantom, using that patch's own mean water and land intensities from the GT labels. I multiply it by Gamma(4.4, 1/4.4) speckle, run every method on the result, and compute PSNR against the phantom in normalised dB with peak = 1. This measures noise suppression on idealised data, not fidelity on real texture.

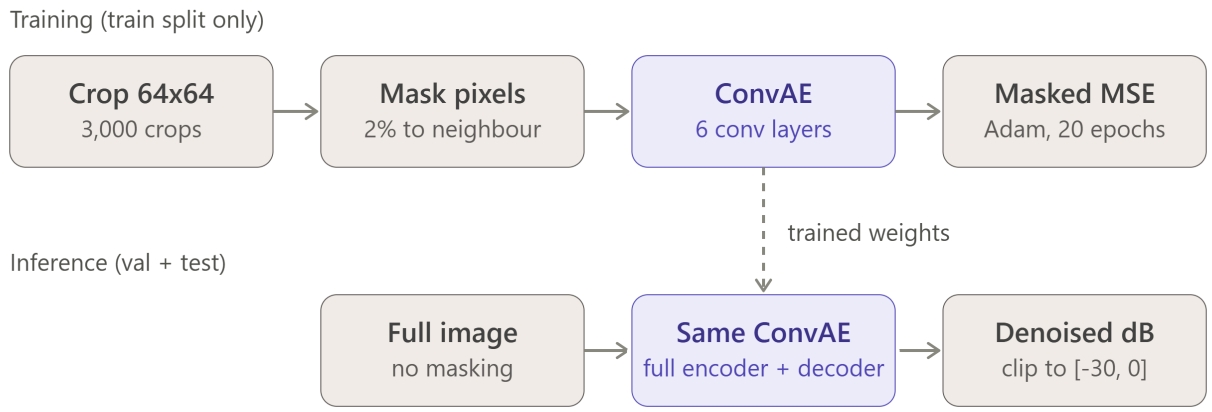
Verification, stated plainly

- Executed here: syntax parsing of all code cells, and the numpy-only filter and metric cells with their sanity checks (constant image preserved; ENL rises on simulated speckle).
- Not executed: data download, the autoencoder training and inference cells, and the evaluation and plotting cells. PyTorch could not be installed in this environment.
- Untested assumptions: the GCS URL layout and CSV format, the nominal ENL of 4.4, and the runtime estimate, which I put at under 15 minutes. All of these need a Colab run.

Here is the end-to-end system design and data flow first. The Noise2Void training logic follows as a second diagram.


Load 100 patches
VV dB + hand labels
Validate + split
dB check, 60/20/20, seed 0
Train split (60)
labels never used
Val (20) + test (20)
val tunes, test reports
Train compact AE
Noise2Void masked MSE
Denoise val + test patches
Filters run on linear intensity
Raw
Median
Lee
Frost
Conv AE
ENL (real)
GT-water blocks
Segment + IoU/F1
-17 dB + val-tuned
PSNR (simulated)
speckled GT phantom
Results table + figure
Learned path
Evaluation
Data and classical steps

Training and Inference logic of the autoencoder :



The key system decisions behind these diagrams are below.

- **Leakage control:** the split is by patch. The autoencoder trains only on the 60 train patches and never sees labels. The 20 val patches tune the segmentation threshold, and the 20 test patches are used only for reporting.
- **Filters on linear intensity:** speckle is multiplicative, so Median, Lee and Frost run on linear values and the results are converted back to dB.
- **Noise2Void masking:** a plain reconstruction autoencoder would learn the identity and keep the speckle. The masked-pixel loss forces it to predict each pixel from its context.
- **Fixed and tuned thresholds:** every method is segmented at the reference `-17 dB` and at a threshold tuned on the val patches.
- **Metrics:** ENL is measured on real ground-truth water blocks. PSNR is measured on a simulated, speckled phantom, because real patches have no clean reference.

These diagrams show the code as written. None of it has been run end to end, so test it in Colab.

Result table interpretation :

1. The autoencoder under-denoises

- Its ENL is 3.08 against 2.83 for raw, and its PSNR is 23.0 dB against 22.5 dB for the noisy input, about +0.5 dB. The classical filters gain 10 to 12 dB.
- This matches the very low masked-pixel loss. The likely explanation is that the speckle in these patches is spatially correlated, so the network learns to copy it from neighbouring pixels. That breaks Noise2Void's independence assumption.
- It is also telling that it fails on the phantom, where I injected independent speckle. A network that learned to copy neighbours would also leave that noise in place. I haven't verified this, so measure it with the check below.

2. Segmentation barely depends on the denoiser at -17 dB

- Fixed-threshold IoU is 0.50 to 0.51 for every method, so denoising does not rescue the reference threshold.
- Tuned thresholds land at -12.5 to -14 dB, far above -17. The reference rule is badly matched to these labels, and it is the main limit on accuracy (F1 about 0.72 at best).
- Tuning alone lifts raw IoU from 0.506 to 0.523. The filters add about 0.04 on top of that (0.565 to 0.571). That gain is real but modest.

3. Method ranking

- Median has the best ENL and tuned IoU, while Frost and Lee have the best PSNR. The tuned-IoU differences among the three classical filters (0.564 to 0.571) are too small to rank on 20 patches.
- ENL is 2.83 on real water, below the 4.4 I assumed, so the real data is not at the nominal look count. Real water also has wind texture, which counts as variance.
- A sanity check passes: the raw PSNR equals the noisy-input PSNR (22.491), as it should.

4. One item I cannot explain

- The validation split shows 19 patches. A 60/20/20 split of 100 patches should give 20. A split of 97 patches would give 58/19/20, which would mean fewer than 100 were loaded, but I can't tell from here.
- Please check `len(VV)` and the "train/val/test" line printed by the data cell. If it is below 100, some downloads or files are missing.

Checks to confirm the correlation explanation (run after the data cells; they only read data):

python

```
def lag1(b): b = b - b.mean(); return (b[:, 1:] * b[:, :-1]).mean() / b.var()
blk = [VV[i][r:r+16, c:c+16] for i in idx_te for r, c in
homogeneous_blocks(LAB[i], np.isfinite(VV[i]))]
print("lag-1 autocorr:", np.median([lag1(b) for b in blk]), " dB std:",
np.median([b.std() for b in blk]))
```

A lag-1 autocorrelation above about 0.3 supports correlated speckle. A dB std well under about 2.2 means the data is smoother than the 4.4-looks assumption.

If correlation is confirmed, the standard fix is a structured blind-spot mask: mask a patch of several pixels, or replace from beyond the correlation length, so the network cannot copy noise

from adjacent pixels. That is an algorithm change, so I won’t touch the notebook until you decide. If you want it, tell me the autocorrelation result and I’ll make that change.